applied sciences

MDPI

Article

Machine-Learning-Based Detection of SQL Injection Attacks in Web Applications

Haya Aldossary and Noura Aleisa *

College of Computing and Informatics, Saudi Electronic University, Riyadh 11673, Saudi Arabia
* Correspondence: n.aleisa@seu.edu.sa

Abstract

The increasing development and prevalence of web applications have contributed to a surge in web attacks, with injection vulnerabilities considered amongst the most pivotal, widespread, and severe. SQL injection attacks are a substantial type of attack that exploits flaws in web applications to carry out malicious SQL commands. This study aims to evaluate the effectiveness of machine-learning algorithms for detecting SQL injection attacks based on query patterns. The proposed approach involves data preprocessing, employing machine learning (ML) algorithms such as XGBoost, AdaBoost, SVM, and Light Gradient Boost, and comparing their performance, which was applied using a publicly available SQL injection dataset from Kaggle. This study used two datasets: one for training and testing the classifier and one for evaluating its performance. The dataset was preprocessed using TF-IDF for feature extraction, and the models were evaluated using standard performance metrics, such as accuracy, precision, recall, and F1-score. Based on the test results, SVM outperforms all other models across all metrics with an accuracy of 0.9847, precision of 0.9847, recall of 0.9847, and F1-score of 0.9874, closely followed by XGBoost. Light Gradient Boost and AdaBoost exhibit lower performance across all metrics, suggesting that SVM and XGBoost are more effective classifiers for the dataset. These findings highlight the effectiveness of machine learning methods, especially SVM, in recognizing SQL injection attacks and enhancing web application security.

Keywords: SQL injection; XGBoost; AdaBoost; SVM; Light Gradient Boost

1. Introduction

The rising demand for web applications has increased the frequency and severity of online attacks [1]. Pattewar et al.'s research indicates that injection is the most common vulnerability in online apps [2]. According to Fang et al., injection attacks take advantage of several weaknesses to allow the input of user data that is not trustworthy, which is then processed by the online application [3]. For instance, it involves inserting malicious SQL commands into input fields or queries to compromise a database's security or manipulate its data [4]. According to Yan et al., most modern online apps are especially vulnerable to these kinds of attacks since they depend on a backend database to store user data and retrieve information upon user request [5].

It is now obvious that network information is proliferating due to the Internet's rapid expansion. Web apps are convenient, but they can raise serious network security concerns. Early large-scale studies, such as Qihoo 360's study, reported that 1.979 million Chinese websites had their security evaluated by the end of 2016, and the results showed that 46.3% of web apps had security flaws. The most significant number of these vulnerabilities

Academic Editor: Jose María Alvarez Rodríguez

Received: 8 March 2026
Revised: 8 April 2026
Accepted: 6 May 2026
Published: 9 May 2026

was caused by cross-site scripting (XSS) and SQL injection (SQLI) attacks [6]. One of the most prevalent network security flaws, SQL injection attacks, requires attention. An SQL injection assault against Sony's PlayStation Network in April 2011 served as an example of the situation. The attack allowed access to over 77 million accounts and resulted in the theft of 12 million credit cards. Among the information exposed were user accounts, passwords, addresses, and credit card transaction histories. As a result, Sony suffered indirect damages of USD 170 million. Similarly, in February 2017, the Russian hacker known as "Rasputin" successfully breached systems by gaining unauthorized access to a database server through the use of SQLI vulnerabilities. Significantly sensitive data from over 20 institutions and government organizations in the US and the UK was stolen because of this intrusion [7]. Despite this study being conducted a decade ago, it highlights the long-standing nature of the issue. According to the Open Web Application Security Project (OWASP) Top 10, injection vulnerabilities remain one of the most critical security threats against web applications, which shows that even recent cybersecurity reports continue to emphasize the severity of injection attacks. This report gives developers essential insights into potential vulnerabilities by outlining the most important security concerns that online applications encounter. In addition, more recent studies ensure that SQLI attacks stay a dominant threat in Web application systems, indicating that this risk persists even though there are improvements in web security [8].

Recognizing the gravity of such security threats, the OWASP contributes significantly to improving software security on a worldwide scale. The "OWASP Top 10," a well-known list that OWASP produces, is meant to increase awareness among web application security practitioners and developers [9]. Similarly, MITRE, a renowned organization in the field of cybersecurity, releases the CWE Top 25 Most Serious Software Vulnerabilities list. SQLI attacks also hold the third position in this ranking, highlighting their persistent and serious nature as software vulnerabilities [10]. These rankings serve as crucial guidelines for developers and security professionals in identifying and mitigating potential security risks associated with web applications, emphasizing the urgent need for robust security measures to combat injection attacks effectively. In a study by Pejo & Kapui, the authors examined the literature on ML-based SQLI, emphasizing four areas: models, datasets, features, and assessment [11]. Forms and cookies are the usual ways that users interact with web applications. By inserting malicious code into user inputs, hackers take advantage of this interaction to generate SQL queries. If user inputs are not properly validated, it can pave the way for successful SQL injection attacks, having serious repercussions, such as the destruction of databases or the theft of confidential customer information via web-based applications [12]. Numerous research efforts have focused on detecting SQLI attacks using machine learning algorithms. While some studies concentrate on detecting SQLI after it has already transpired, others aim to proactively prevent such attacks from occurring in the first place. Regarding SQLI defense mechanisms, the integration of machine learning is poised to significantly enhance detection capabilities in the future. Nevertheless, there remains a gap in the availability of white box detection techniques rooted in machine learning, as many researchers lean towards utilizing predefined algorithms within integrated tools. A recent paper on network security and privacy protection against emerging threats has discussed enhancing data confidentiality and resilience. The study proposed using advanced encryption mechanisms to safeguard sensitive data in digital environments [13]. This progress highlights the demand for comprehensive security solutions that include the use of both data protection and attack detection mechanisms in web applications, such as SQL injection detection.

While traditional techniques, such as input sanitization using regular expressions, are considered the fundamental defense mechanism against simple attacks for detecting

SQL injection patterns, their effectiveness relies on accurate and consistent implementation. These methods may be insufficient, misconfigured, or easily bypassed due to the complex input structures and dynamic query generation [7,14]. In addition, attackers always develop new strategies to generate obfuscated or unseen injection patterns that may not be detected by predefined rules, which means that relying on rule-based validation only is not sufficient. In this context, ML-based methods can improve traditional security methods by identifying abnormal or malicious behavior beyond predefined rules [15,16]. ML-based methods can automatically learn patterns in queries and distinguish anomalies or malicious behavior that may not be detected by defined rules. As a result, machine-learning-based methods can provide a better adaptive and scalable approach to detecting SQL injection attacks in web applications [17]. Previous studies show that traditional methods struggle to catch evolving SQL injection techniques effectively, and using inputs embedded within dynamically constructed SQL queries or user-generated content fields cannot consistently be constrained with simple validation rules. In contrast, ML-based techniques highlight their better adaptability, as they have shown high detection performance, with accuracy above 96% in determining both seen and unseen SQLI attacks [18]. The main research questions addressed in this study are: (1) How can machine learning models detect SQLI attacks effectively in comparison to traditional methods? (2) Which machine learning model achieves the optimal performance for SQLI detection? (3) How does TF-IDF as a feature representation affect the classification performance?

Aim and Contributions

The research's goal is to detect SQLI attacks using machine-learning algorithms to protect databases. In the proposed system, researchers use advanced algorithms like XGBOOST, DABOOST, SVM, and Light Gradient Boost to detect SQL attacks. Finally, the models are compared, resulting in an accurate approach to preventing injection attacks and better results. This research proposes a machine-learning-based framework for detecting SQL injection attacks in web applications using query representation and natural Language Processing (NLP) techniques. Previous studies focus on a single classifier, but this research provides a comparative analysis of multiple machine learning algorithms (SVM, XGBoost, LightGBM, and AdaBoost) using a unified preprocessing pipeline and evaluates their performance on multiple datasets. This research also entails extensive testing and refining of the model's performance, as well as its smooth integration into existing web environments. Ultimately, the study concludes by documenting its findings and presenting an economic analysis, along with providing guidelines for future research directions.

2. Background and Related Work

The review explores the integration of machine learning algorithms to detect SQL injection attacks in web applications. It delves into the limitations of traditional prevention methods and highlights the role of machine learning in enhancing detection capabilities. The review looks at SQL injection attacks, old methods of detection, and new improvements in machine-learning-based methods of detection. The goal is to identify the most pressing issues and opportunities for further research in this critical area of cybersecurity.

2.1. SQL Injections

According to Sermpinis, SQL injection is a database exploitation technique in which a hacker manipulates data input within an application to enter an SQL query and obtain access to records within an information system. SQL injection techniques can be classified into three categories: out-of-band, in-band, and blind SQL injection [19]. Attackers who use in-band SQL injection, which can be error- or union-based, use the same communication

channel to send and receive data. On the other hand, out-of-band SQL injection occurs when query replies are obtained via an unrelated channel, most frequently FTP, HTTP, or DNS. Finally, blind SQL injection depends on the attacker exploring the behavior of the server instead of seeing the response directly. It can be either content-based or time-based [11]. Penetration testers typically utilize SQL injection tools as valuable resources to evaluate the security posture of websites owned by businesses. These automated tools streamline the execution of SQL injection attacks, sparing security professionals the need to develop them from scratch. Kalilinuxtutorials.com provides a comprehensive list of top-tier open-source SQL injection tools, such as Sqlmap, Jsql Injection, BBQSQL, Nosqlmap, DSSS, and others. Different features are offered by each instrument. By offering automatic SQL injections, SQLMap enables users to breach databases without requiring them to understand the intricate coding of the statements [20].

2.2. Machine Learning to Detect SQL Attacks

Machine learning operates on algorithms capable of learning from data without the need for explicit rules-based programming [21]. This idea involves using machine learning techniques to give machines the ability to make decisions on their own. Thus, it circumvents the necessity for traditional programmed instructions. Another study has carefully examined, with an emphasis on the training and testing stages, the application of artificial intelligence for both attacking and defending against security attacks. They separated advancements and uses of ill-disposed attacks in their exploration according to how they connect with PC Vision2 regular language processing: cyberspace security and this present reality. In addition, during the process of their research, the authors discussed defensive strategies and offered suggestions for dealing with specific types of aggressive attacks [22]. In 2019, Pan et al. looked at over 200 IS examinations using opposing machine-learning strategies in models for finding malware and interruptions. The authors of the report outlined the most prevalent adversarial attacks along with defenses against malware and interruptions [1]. Dasmohapatra & Priyadarshini conducted an extensive examination of SQLI attacks, encompassing their methods, detection mechanisms, and preventive strategies. The authors elucidated how perpetrators exploit vulnerabilities to execute malicious code and outlined approaches to mitigate the adverse impact on database systems. Given that web operations often entail online administration tasks like informal communication, transaction account management, and handling sensitive user data, unauthorized access poses a significant risk, exposing data to potential attacks. Attackers use a range of malicious hacking and cracking methods to compromise systems. They could use crafty queries and strategies to bypass authentication controls and take complete control of the online application and server. Developers have responded to these dangers by creating a multitude of sophisticated algorithms that encrypt data searches and strengthen defenses against these types of assaults by putting strong query modification tactics into place [23]. More than 14 researchers employ Convolutional Neural Networks (LSTM-DBN, NILP, and BiLSTM) as well as other deep-learning techniques to identify SQL injection threats [24]. They also included a method comparison in terms of datasets: characteristics, methodologies, and aims. A total of 82 studies were considered [2]. They studied and analytically assessed the techniques and tools often used to identify and stop SQL injection attacks. Their review revealed that most scholars focused on proposing strategies to detect and prevent SQLI attacks rather than assessing the effectiveness of current SQLI attack detection methods. Another study demonstrated the issue through a decision-making cycle. Support learning specialists can conduct security examinations and entrance testing at a later stage, as indicated by the trial findings. They display SQL queries as a collection of tokens, and they use the token centrality percentage to create both

single and multiple SVM classifiers. The after-effects of the examinations showed that the recommended technique could effectively distinguish malevolent SQL questions [25]. In 2021, Abaimov and Bianchi shared a two-class support vector machine (TCSVM) model that could tell if an SQL injection attack in a web request would succeed or fail. This method utilized ML predictive analysis to anticipate SQL injection dangers by blocking web requests at the intermediary level [12]. In a study by Shwaish et al., the researchers provided an inventive strategy for classifying SQL queries. They determined the closeness measure between the query strings using the whole-weighted-string after-effect portion approach. The researchers then developed a support vector machine based on likeness measurements to determine whether the query strings were genuine or pernicious. They used a few datasets to evaluate the proposed strategy, achieving a precision rate of 92.480 [26].

2.3. Related Work

The study considered several papers that were gathered via database searches using keywords, such as "SQL Injection" + "Machine Learning", and forward and backward snowballing from the surveys. Abaimov & Bianchi conducted a review focusing on SQLI prevention in web applications. They provided an overview of 14 unique SQLI attack types and how they affect web applications. Their main goals were to examine several approaches for preventing SQL injection attacks and evaluate the best defenses against them [12]. Researchers in another study by Hasan et al. presented, tested, and compared the performance of 23 ML classifiers for detecting SQLI attacks using MATLAB. They created custom datasets containing abnormal SQL syntax injections and manually verified the SQL statements. To train the classifiers, 616 SQL statements were used in total. A variety of machine learning algorithms were applied, including logistic regression, cubic SVM, fine Gaussian SVM, cosine KNN, complex tree, cubic KNN, linear discriminant, medium tree, subspace KNN, simple tree, quadratic discriminant, and SVM. The study determined that the five most accurate models were ensemble-boosted, bagged trees, linear discriminant, cubic SVM, and fine Gaussian SVM. These models achieved an accuracy rate of 93.8% for detecting SQLI attacks [27].

Additionally, Wang & Li proposed a hybrid approach employing tree-vector kernels in Support Vector Machines (SVMs) to learn SQL statements. They leveraged both the parse tree structure of SQL queries and the similarity characteristics of query values. Results demonstrated the effectiveness of their approach in efficiently and accurately identifying abnormal queries [28]. Kar et al. devised a method to train a Support Vector Machine (SVM) in identifying malicious SQL queries by transforming the WHERE clause of the query into a network of interaction tokens and determining the most central nodes within it. Node centralities were utilized to measure the importance or centrality of each node in the network. Experimental findings affirmed that the three centrality measures employed in their study effectively detected SQLI attacks with minimal impact on performance [29].

Zhang's paper has introduced a machine-learning classifier designed to identify SQLI vulnerabilities in PHP code. Various machine learning techniques, including logistic regression, Random Forest, Support Vector Machines (SVM), Long Short-Term Memory (LSTM), MultiLayer Perceptron (MLP), and Convolutional Neural Networks (CNN), were trained and evaluated. The study found that CNN yielded the highest precision of 95.4%, whereas an MLP-based model achieved the highest recall at 63.7% and the highest f-measure at 74.6% [30]. Li et al.'s study has proposed an LSTM-based method for detecting SQLI attacks, capable of automatically learning optimal data representations, and particularly adept at handling large volumes of complex, high-dimensional data. Additionally, from a penetration testing perspective, the paper introduced a method for generating injection samples based on data transmission channels. This approach produced legitimate positive samples

and accurately simulated SQLI attacks, effectively addressing overfitting concerns resulting from insufficient positive samples. Experimental results demonstrated that the proposed method outperformed many commonly used deep learning (DL) and classical machine learning (ML) techniques in terms of identifying SQLI attacks and reducing false positives. Moreover, the method achieved accuracy, precision, and f-score metrics all-surpassing 92% [31].

Pathak et al.'s work has employed a progressive neural network model with an NB classifier to effectively identify SQLI attacks. They trained the progressive neural networks on parameters associated with various SQLI attacks, including error-based, time-based, SQL query, and union-based attacks. The proposed method achieved an impressive accuracy rate of 97.897% [32]. Ahmed and Uddin proposed the utilization of natural language ensemble learning and processing to develop a bag-of-words model for SQLI detection, utilizing a Random Forest classifier. The study also incorporated prediction enhancement to improve the classifier's detection capability. Additionally, KNN, NB, SVM, and DT classification models were trained on the same testing dataset, and their performances were compared with the proposed method. Results indicated that the proposed approach outperformed the other classifiers in terms of accuracy, True Positive Rate (TPR), and False Negative Rate (FNR). Evaluation metrics such as confusion matrix, precision, true-negative rate, false-positive rate, receiver operating characteristic curve, and area under the curve were utilized to measure classifier performance [33].

Tripathy et al.'s paper has introduced a novel method for detecting SQLI attacks using human agent knowledge transfer (HAT) and transfer learning (TD ML). In this model, an ML agent was trained as a maze game to distinguish between normal and malicious SQL queries. The ML agent received more rewards for correctly identifying SQLI attack queries, ultimately achieving an accuracy of 95% [34]. Hu et al. have conducted a survey focusing on SQLI attack detection and prevention, aiming to enhance understanding of SQL and its associated risks among both laypeople and professionals. The research provided insights into ongoing issues concerning web application security and strategies to mitigate SQLI attacks. The authors anticipated that adherence to the strategies outlined in their study could help ensure the safety of online applications developed by web application developers [35]. Mejia-Cabrera et al. have introduced a novel approach for constructing a dataset utilizing a NoSQL query database. The researchers trained and assessed six classification techniques, including Random Forest, Support Vector Machines (SVM), Decision Trees (DT), K-Nearest Neighbors (KNN), Neural Networks, and Multilayer Perceptron, to detect SQLI attacks. Experimental findings indicated that the latter two techniques achieved an impressive accuracy of 97.6% [36]. Chen's study has proposed a deep learning (DL)-based method for detecting and preventing SQLI attacks. Drawing upon extensive domestic and international research, the author suggested an SQLI detection technique leveraging NLP and DL frameworks, obviating the need for a predefined rule base. This strategy enhanced accuracy, reduced false alarm rates, and effectively guarded against previously undetected attacks by enabling automatic identification of the language model characteristics associated with SQLI attacks [4].

Alghawazi et al. have conducted a systematic literature review encompassing 36 articles of research on SQLI attacks and ML techniques. They identified the most commonly utilized ML techniques for classifying different types of SQLI attacks. Their findings highlighted a scarcity of studies utilizing ML tools and techniques to generate new SQLI attack datasets, as well as a limited focus on employing mutation operators for generating adversarial SQLI attack queries. The researchers aimed to address these gaps in future work by exploring additional ML and DL techniques for generating and detecting SQLI attacks [37].

Zhang et al. have addressed the challenge of detecting SQLI statements, a common and formidable network attack known to cause significant disruption and data loss. They developed an SQLI detection model and technique leveraging the properties of SQL statements. Their approach involved converting data into word vectors through word pausing, constructing a sparse matrix, and training a multi-hidden-layer deep neural network model using the ReLU function. The model achieved an accuracy exceeding 96%, effectively mitigating overfitting issues and obviating the need for manual feature extraction. This approach significantly improved SQLI detection accuracy compared to conventional machine learning and LSTM algorithms [24].

Muhammad & Ghafory have explored various ML methods to address the challenge of detecting vision impairment SQL injection attacks. Their study, conducted as part of the Waikato Climate Data Exploration, investigated the performance of several supervised learning classification algorithms, including Bayes Network (BNK) Stochastic Gradient Descent (SDG), Sequential Minimal Optimization (SMO), Logistic Regression (LRN), Instance-Based Learner (IBK), Naive Bayes (NB), J48, and MultiLayer Perceptron (MLP). The evaluation was conducted using both 10-fold cross-validation and a hold-out method with a 70/30 split. Results revealed that SMO, IBK, and J48 achieved high accuracy values of 98.7785%, 98.4285%, and 98.2985% with cross-validation, and 98.7956%, 98.1526%, and 100 with the hold-out method, respectively. SMO emerged as the top performer, exhibiting effectiveness in model training, cross-validation, and overall performance, making it the preferred choice for detecting and preventing SQL injection attacks. Accuracy, along with other performance assessment indicators, played a crucial role in selecting the most suitable algorithm for predictive analytics [38].

Crespo-Martínez's work has introduced models for detecting SQL injection attacks in flow data, aiming to enhance user, company, or administration security by generating alerts based on network flow data analysis. LR and Perceptron+SGD models demonstrated promising results, achieving accuracies exceeding 96% and a False Alert Rate (FAR) of less than 1%. Additionally, the model combining various techniques (VC) exhibited a high SQL injection attack detection capability, with an accuracy score of 85.6% and a Detection Rate (DR) of 89%. These findings underscore the potential of NetFlow as a lightweight, flow-based protocol for detecting SQL injection attacks in networks [7]. Alarfaj & Khan have utilized a dataset comprising 6000 SQL injections and 3500 normal queries, extracting features through tokenization and regular expression-based techniques and selecting them using Chi-Square testing. Their proposed Probabilistic Neural Network (PNN) demonstrated strong performance, achieving an accuracy rate of 99.19%, precision of 99.5%, recall of 98.1%, and F-measure of 92.8% when evaluated using a 10-fold cross-validation approach. These results highlight the efficacy of the proposed PNN classifier in accurately detecting SQL injection attacks across various scenarios [39].

A recent study by Lakshmi has explored the use of machine learning techniques to improve the detection of SQL injection in web applications. The study proposed an ML-based approach that analyzes patterns in query structures and user inputs to detect malicious SQL queries and stop database attacks. The results show a significant enhancement in detection accuracy and database security against SQL injection attacks [40].

Another study by Thilakraj has continued to use ML approaches to enhance SQL injection detection in web applications. The study proposed an intelligent detection approach to analyze the behavioral patterns in SQL query structures to detect malicious database commands. The proposed method classifies queries and detects injection attacks more effectively than traditional rule-based techniques. The experiment results show improved detection accuracy against SQL injection threats [41].

A recent study by Al Mallah in 2025 has also explored advanced machine learning techniques for detecting SQL injection attacks. The study proposed an ML-based approach for detecting SQL query attacks by analyzing database requests to identify query and behavioral patterns. The study effectively distinguishes between normal and malicious SQL queries, achieving improved detection accuracy compared to traditional rule-based techniques. The findings show the importance of intelligent detection mechanisms for web application security and protecting databases from SQL injection threats [42].

Table 1 provides a comparison of some existing studies on SQL injection detection. It can be observed that the studies are focused on particular models or datasets, with limited generalization. Additionally, some studies lack comprehensive comparative evaluations, and they rely on manually built datasets. These limitations highlight the gap for a systematic evaluation of machine learning models.

Table 1. Comparative analysis of selected studies on SQL injection detection.

Study	Approach	Dataset	Key Contribution	Limitations
[12]	Review of SQLI prevention techniques	Not specified	Overview of 14 SQLI attack types and prevention methods	Lacks experimental evaluation and comparative analysis
[27]	Multiple ML classifiers (23 models)	Custom dataset (616 SQL queries)	Comparative evaluation of many classifiers using MATLAB	Small dataset size; manual dataset creation
[28]	SVM with tree-vector kernels	Not specified	Uses SQL parse tree + query similarity for detection	Dataset not specified; may have high complexity
[29]	SVM with graph-based features	Not specified	Uses token interaction network and centrality measures	Computational complexity; feature engineering dependent
[30]	ML classifier for PHP vulnerability detection	Not specified	Detects SQL injection in PHP code	Limited scope (code-level only)
[26]	SVM with string similarity measures	Multiple datasets	Classifies SQL queries using weighted string similarity	Moderate accuracy (92%); feature-specific approach
[7]	LR, Perceptron, ensemble models	NetFlow data	Detects SQLI using network flow analysis	Lower accuracy in some models; indirect detection
[39]	Probabilistic Neural Network (PNN)	6000 SQLI + 3500 normal queries	High accuracy (99.19%) using feature selection (Chi-square)	Dataset-specific performance; generalization unclear
[40]	ML-based pattern analysis	Not specified	Detects malicious queries using query structure patterns	Details of dataset and model not specified
[41]	Intelligent ML detection	Not specified	Improves detection over rule-based methods	Limited methodological details
[42]	ML behavioral analysis	Not specified	Detects SQLI using query behavior patterns	General description; lacks detailed evaluation

The reviewed works disclose several limitations. Several studies focus on only one model or dataset, which limits the generality of applying it to different scenarios. Additionally, other studies depend on manually constructed or small-scale datasets, which may not represent real-world SQLI. Moreover, there are limited studies that have conducted systematic comparative evaluations under a unified experimental framework. These gaps required a comprehensive assessment of machine learning models using standardized preprocessing and consistent evaluation settings.

3. Methodology

This research outlines the steps taken in preparing and applying the data, including fundamental tasks such as loading the dataset, analyzing and visualizing it, preprocessing it, and splitting it. The study involves the machine learning algorithms by utilizing XGBoost, AdaBoost, SVM, and Light Gradient Boost, which will also be explained, and how they were applied to the data. Figure 1 presents an overview of the study's system architecture.

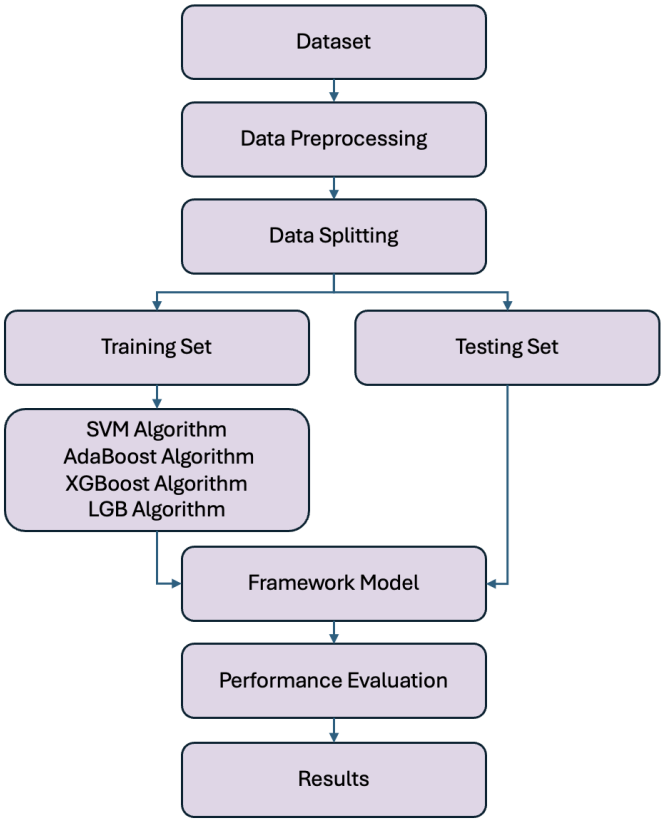

Figure 1. Overview of the proposed machine learning framework for SQL injection detection, illustrating the main stages including data preprocessing, model training (SVM, XGBoost, AdaBoost, and LightGBM), and performance evaluation.

3.1. Dataset

The dataset was gathered, encoded, and then translated into the CSV file format as part of the system design. In this study, the primary dataset was collected from publicly available SQL injection datasets hosted on Kaggle. The Kaggle dataset used is the [SQL injection dataset] published by Syed Saqlain Hussain on Kaggle, which includes queries labeled as either malicious (SQL injection) or benign (normal queries). This dataset was selected in this study because of its public availability and labeled structure. The dataset is pre-labeled by its original authors, which made it suitable for training and testing machine learning models for SQLI detection. Accordingly, the labeling process, involving annotator details, annotation guidelines, and inter-annotator agreement, was not necessary in this study. The dataset was manually prepared, comprising a total of 4199 SQL injection attack queries along with normal text. Different SQL injection queries were collected and labeled as either 'NORMAL' or 'SQLI' classes, such as the use of tautologies, UNION-based injections, and comment-based manipulations. The dataset's statistics are illustrated in Figure 2 [43].

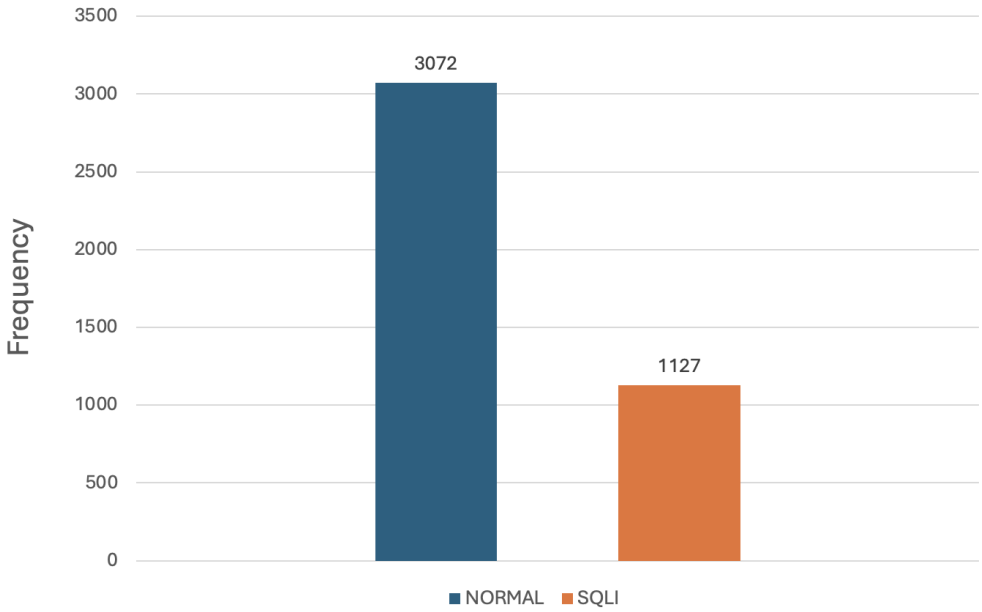

Figure 2. Distribution of SQL injection and normal queries in the dataset, illustrating the class balance between malicious and benign samples used for model training and evaluation.

The Kaggle dataset contains plain-text sentences generated from payloads received from HTML forms. This dataset presents a blend of URLs, special characters, textual data, and numerical data. This diversity in data types is beneficial for training the model to accurately identify SQL injections while minimizing false positives. Several features of this dataset render it particularly suitable for addressing the problem at hand. The dataset is generated by gathering user inputs from a web application form. This source of data enhances the likelihood of encompassing a wide array of scenarios, thereby facilitating the efficient training of the model. In general, the dataset consists of two primary types of data: normal text and SQL injection queries. The Kaggle dataset will be divided into two segments: training data and test data. The split ratio used in this study is 7:3, where the larger portion is allocated for training, and the remaining portion for testing [43].

The UCI dataset was employed as an independent evaluation dataset to test the generality of the trained models. The UCI dataset has two labeled SQL queries—malicious (SQL injection) and benign (normal queries). This dataset is also publicly available, has a structured format, and is appropriate for SQLI detection assignments. In this study, the trained models were directly evaluated on the UCI dataset, which served as an external validation dataset to evaluate the models' performance on overlooked data with various characteristics. This setup ensures that the results of model evaluation reflect the robustness and generality of the ML models. Unlike the Kaggle dataset, which was employed for both training and internal testing via a train–test split, the UCI dataset was not involved in the training process [44].

3.2. Data Preprocessing

It is important to note that the specific pre-processing steps may vary depending on the characteristics of your dataset and the machine learning algorithm you are using. However, some common pre-processing steps for text data include tokenization, encoding labels, stemming and lemmatization, and vectorization. The SQL queries were converted into numerical representations using the Term Frequency–Inverse Document Frequency (TF-IDF) vectorization technique, by first splitting queries into individual tokens, then by applying preprocessing steps such as stemming and removal of unwanted characters.

In preprocessing, the SQL queries were converted into a structured dataset to ease the feature engineering and analysis. Hashing was used to analyze the SQL queries into meaningful symbols according to their structure and rules. This step enables the distinction between legitimate and malicious queries by identifying the key elements, such as SQL keywords, operands, and special characters, which are essential. Features were extracted using the Term-Did-Frequency Inverse (TF-IDF) before being fed into the machine learning models by assigning weights to symbols according to their significance within the dataset. This approach focuses on SQL keywords that indicate the existence of injection patterns, while minimizing the less significant terms. This ensures that the most informative query components contribute to model training.

3.3. Data Splitting

The collected data was divided into two main categories: training and testing. The training data is utilized to train the model, while the testing data is reserved for evaluating the model's performance on unseen data. During the training phase, the model is developed and optimized using the training set, often involving parameter tuning to enhance its effectiveness. Subsequently, the test set is employed to assess how well the trained model generalizes to new, unseen data. In this study, a standard data-splitting ratio of 70% for training and 30% for testing was employed.

3.4. Algorithms Selection

In this study, the selection of machine learning algorithms, specifically Support Vector Machine (SVM), XGBoost, AdaBoost, and Light Gradient Boosting Machine (LightGBM), was based on several factors, including effectiveness in classification, processing high-dimensional textual data, and computational efficiency. Another factor was their widespread use in intrusion detection and cybersecurity [45]. A popular supervised learning model for regression and classification applications is the Support Vector Machine (SVM). It is preferred over other models because it can get great accuracy with less complexity and computing power. In addition, SVM has strong performance in high-dimensional feature spaces and its ability to identify the best decision boundaries. SVMs have applications in several domains, such as intrusion detection and computer security. In the context of computer security, SVM, including variations like one-class SVM, has been employed for analyzing records based on novel kernel functions [46] and achieving accurate Internet traffic classification [47,48]. XGBoost stands out as an efficient and adaptable ensemble classifier for gradient boosting. In its segmentation process, this classifier often employs a greedy technique to optimize existing nodes. Sequential trees are typically built upon previous ones, aiming to minimize inaccuracies from prior iterations. This is achieved through the incorporation of a regularization term, streamlining the error reduction process [49]. An AdaBoost classifier is a machine learning classifier that improves its predictive performance by fitting numerous instances of the classifier on the same dataset after it has first been fitted on the original dataset. One of the key features of AdaBoost is its ability to focus on the most influential features, thereby improving its overall effectiveness. This emphasis on important features is considered crucial for achieving high performance, as highlighted in the literature [50]. For regression and classification tasks, a machine learning technique known as gradient boosting is utilized, which builds a prediction model from an ensemble of weak prediction models—usually decision trees. To iteratively enhance the model's performance, it combines an additive model, a weak learner, and a loss function. The process involves adding weak learners to minimize the loss function by leveraging the patterns in residuals. This iterative approach strengthens the model with weak predictions, enhancing its accuracy. The underlying principle of gradient boosting is to identify and

exploit residual patterns, continuously refining the model until the residuals exhibit no discernible pattern that can be further modeled. This prevents overfitting, ensuring that the model generalizes well to unseen data [51]. Ensemble methods such as XGBoost, AdaBoost, and LightGBM were selected in this study due to their capability to enhance predictive performance by merging several weak learners and lower overfitting, and because they are widely adopted in previous studies on SQLI detection and security tasks [48,52].

3.5. Feature Selection

Feature selection is a critical iterative process aimed at identifying the most relevant features or eliminating variables that do not contribute to the model's ability to map the relationship between data variables and target outcomes. The selection of optimal features is crucial for accurately representing the entire problem and achieving the study's objectives. The quality of selected input features significantly influences the expected outcomes of algorithms tailored to the problem at hand. By reducing the number of irrelevant features, the training time of the model can be shortened, performance can be enhanced, and the complexity of the algorithm can be minimized [53].

3.6. Evaluation

Evaluation measures like accuracy, precision, recall, and F1-score are used in computational tasks like detection and classification to forecast class instances and identify the corresponding categories. These metrics, which are derived from classification approaches, provide important information about the capabilities and performance of the model.

Model Performance Metrics: F1-score, accuracy, precision, recall, and area under the ROC curve (AUC) are common evaluation metrics. Cross-Validation: To make sure the results are reliable and resilient, it is important to employ cross-validation techniques like k-fold cross-validation. Using various combinations of training and validation sets, the model is trained k times, resulting in the partitioning of the training data into 5-k subsets. The performance metrics are then averaged over the k iterations. To ensure the reliability and robustness of the evaluation, k-fold cross-validation was implemented with k = 5, where the dataset was divided into five subsets and the model was trained and validated repeatedly. This approach helps to reduce the performance estimation variability and enhance the generalization. The performance values represent the mean estimates received from five-fold cross-validation. Mainly, the dataset was split into about equal-sized five folds. In each run, four folds were used for training, and one fold for testing. This process was repeated five times, where each fold was performed once as the test set. The evaluation metrics were measured for each run, and the final values were acquired by calculating the mean of these five results. This process delivers a more stable and reliable model performance estimation and reduces the effect of data partitioning variability. Due to computational constraints, standard deviation measures are considered as part of future work.

Hyper-parameter Tuning Results: Experimenting with various hyperparameters for each algorithm should result in a set of optimal hyperparameters for each model. The performance of the models with different hyper-parameter configurations can be compared based on the chosen evaluation metrics.

Model Selection: The best-performing model can be chosen using preset criteria once all models have been assessed using the training set. Depending on the precise objectives of the study, this could be the model with the highest accuracy, the F1-score, or another pertinent statistic.

Testing Phase Evaluation: Once the best-performing model is selected, it needs to be evaluated using the unseen data from the testing set. The same performance met-

rics calculated during training can be used to evaluate the model's performance on the testing data.

Comparison of Results: The best method for the given dataset and problem can be found by comparing the outcomes of all the models. Furthermore, an assessment of the chosen model's capacity to generalize to new, untested data is provided by how well it performs on the testing data [54].

4. Implementation and Results

In the study, several machine learning algorithms were employed for the detection of SQL injection attacks, including Support Vector Machine (SVM), XGBoost, and Light Gradient Boost. The implementation of the project follows the stages outlined in the previous section. The process involved selecting suitable machine learning models, including SVM, XGBoost, Light Gradient Boost, and AdaBoost, configuring these models to optimize performance, acquiring and preprocessing datasets to ensure their compatibility with the chosen algorithms, training the models using the prepared data, rigorously assessing their efficacy through various evaluation metrics, and ultimately deploying them within operational environments to actively monitor and respond to potential security breaches effectively. The study was conducted using Python 3.10 on Google Colab (cloud-based environment, accessed in 2024), utilizing libraries including Scikit-learn, Pandas, NumPy, and XGBoost, and hyperparameters for each classifier were tuned using grid search. Hyperparameter tuning involved experimenting with different parameter configurations for each model to find the best settings. The parameter selection was according to model performance during cross-validation. The evaluation approach involved both cross-validation and a hold-out approach. The models were trained using the training set and then validated using a 70:30 train–test split. The performance values presented the mean results of five-fold cross-validation.

4.1. Importing Python Libraries

Various Python libraries were imported and used for data analysis, machine learning, and anomaly detection. Libraries like pandas and Numpy are utilized for data manipulation and numerical operations. Scikit-learn is used to split data into training and testing sets. To support classification tasks, tree-based models were used within the machine learning framework. To measure the model's predictive effectiveness, the performance was evaluated using standard metrics such as accuracy. The XGBoost and LightGBM libraries were used to implement the corresponding ensemble models.

4.2. Loading Datasets

The dataset was imported from a CSV file into a structured data frame to ease the preprocessing and analysis approaches. Data handling methods were applied to clean, organize, and prepare the dataset for feature extraction and model training. This ensured that the data was properly prepared for the following machine learning processes [43,44].

4.3. Data Pre-Processing

The pre-processing steps described in Section 3.2 were implemented to prepare the dataset for model training, including tokenizing text into individual tokens or words using various techniques, such as whitespace tokenization, regular expression tokenization, preprocessing steps such as stemming and removal of irrelevant characters, and transforming textual data into numerical vectors using the TF-IDF vectorization technique.

4.4. Data Splitting

A 70–30% split was used in this study for testing and training, respectively, following the methodology described in Section 3.3. Dividing the data is an essential phase in machine learning, where the Kaggle dataset is segmented into various subsets to assess model performance and avoid overfitting. In the study of detecting SQL injection attacks, several machine learning algorithms are utilized, such as Gradient Boosting, XGBoost, AdaBoost, and SVM, using the processed feature vectors. Data splitting ensures the model's ability to generalize effectively to new data. Hyperparameter tuning was performed to optimize model performance, and different configurations were evaluated to present the best-performing model. The last model was then evaluated using unrevealed test data to test its generalization capability.

4.5. Training Machine Learning Algorithms

SVM: The SVM model was trained using the training dataset, and its performance was evaluated on the testing data. The accuracy of the model on the testing data is printed to the console. The Support Vector Machine (SVM) classifier's performance was assessed on a test dataset using various metrics. The SVM exhibited a notably high accuracy, achieving 96.49% accurate predictions across all instances. Moreover, the F1 Score, which balances precision and recall, was also impressive at 0.9649, indicating effective model performance. The results indicate that SVM effectively distinguishes between SQL injection and normal queries and is well-suited for this specific classification task.

XGBoost: An XGBoost model was applied to classify SQL queries as either malicious or benign. Malicious queries represent SQL injection, which tries to manipulate query structure to violate database vulnerabilities, while benign or normal queries represent legitimate database requests. The model was trained to utilize the preprocessed dataset and evaluated on a separate testing set. Using standard evaluation metrics, such as accuracy, the performance was assessed. XGBoost has exhibited remarkable performance on the test dataset. The algorithm achieved an accuracy score of 0.9846. The F1 score, a metric that balances precision and recall, stands at 0.9798. These findings underscore XGBoost's robustness and reliability, making it a suitable model for this specific dataset.

Light Gradient Boost: The Light Gradient Boost (LGB) model was trained and evaluated on the dataset. LGB has exhibited robust performance across multiple metrics, encompassing accuracy, F1 score, sensitivity, specificity, and precision. With an accuracy score of 0.9849, the model accurately predicted 98.49% of cases. An F1 score of 0.9664 indicates a favorable balance between precision and recall. Overall, the performance of the LGB model that uses LGB data has yielded promising results, underscoring its efficacy in predicting both positive and negative cases.

5. Result Analysis and Discussion

Study analysis encompasses evaluating algorithms based on precision, accuracy, recall, and F1-score metrics. To train and assess diverse machine learning classifiers like SVM, XGBoost, Light Gradient Boost, and AdaBoost, cross-validation served as the evaluation method. The study determined the optimal feature count by applying mutual information as a feature selection technique.

5.1. Analysis of Classifiers on the UCI Dataset

A thorough comparison was conducted to assess the performance of SVM, XGBoost, Light Gradient Boost, and AdaBoost. The evaluation results were gauged using four metrics: precision, accuracy, F1-score, and recall [44]. The following Table 2 summarizes the results.

Table 2. Performance comparison of machine learning classifiers (SVM, XGBoost, AdaBoost, and LightGBM) in terms of accuracy, precision, recall, and F1-score on the SQL injection dataset (UCI repository) using TF-IDF feature representation.

Classifiers	Mean Accuracy	Mean Precision	Mean Recall	Mean F1-Score
SVM	0.9808	0.9809	0.9808	0.9807
XGBoost	0.9705	0.9705	0.9705	0.9704
Light Gradient Boost	0.9649	0.965	0.9649	0.9649
AdaBoost	0.6661	0.7981	0.66601	0.6244

The comparison in Table 2 presents the performance metrics of four classifiers: SVM, XGBoost, Light Gradient Boost, and AdaBoost. Each classifier's mean precision, accuracy, F1-score, and recall are displayed. SVM outperforms in all metrics, closely followed by XGBoost. Light Gradient Boost and AdaBoost exhibit lower performance across all metrics, suggesting that SVM and XGBoost are more effective classifiers for the dataset.The difference in accuracy compared to XGBoost (0.97045) is relatively small, even though SVM achieved the highest accuracy (0.98075), which implies that both models perform competitively under the same experimental requirements. Additional visualization methods, such as ROC curves and confusion matrices, can present deeper insights into models' performance and error distribution. These analyses are considered valuable extensions for future work.

5.2. Comparative Analysis of Classifiers on Kaggle (2020)

When analyzing the results on the SQL Injection Dataset (Kaggle) [43], it is evident from Table 3 that the hybrid classifier performed the best in terms of precision, accuracy, F1-score, and recall.

Table 3. Performance comparison of machine learning classifiers (SVM, XGBoost, AdaBoost, and LightGBM) in terms of accuracy, precision, recall, and F1-score on the Kaggle SQL injection dataset using TF-IDF feature representation.

Classifiers	Mean Accuracy	Mean Precision	Mean Recall	Mean F1-Score
SVM	0.9847	0.9847	0. 9847	0.9874
XGBoost	0.9749	0.9757	0.9757	0.9757
Light Gradient Boost	0.9654	0.9623	0.9622	0.9622
AdaBoost	0.8473	0.8675	0.8473	0.8451

Table 3 shows that the SVM classifier emerges as the top performer, boasting the highest mean accuracy and mean F1-score, underscoring its overall superiority. Additionally, its substantial mean precision underscores its reliability in correctly predicting positive examples. Following closely behind, the XGBoost classifier demonstrates a commendable performance, particularly reflected in its second-highest mean F1-score. However, its slightly lower mean recall suggests a potential for overlooking some positive examples. Meanwhile, the Light Gradient Boost classifier excels in mean recall, showcasing its proficiency in identifying positive examples. Yet, its comparatively lower mean precision implies a tendency toward generating more false positives. Bringing up the rear, the AdaBoost classifier lags in all metrics, indicating its suboptimal effectiveness for the specified task. The performance values illustrate the mean results received from five-fold cross-validation. Variability measures, such as measuring standard deviation, were not included in this study.

It is worth noting that classifier performance can vary based on dataset specifics and feature intricacies. Moreover, hyperparameter tuning and feature engineering play pivotal roles in shaping classifier efficacy.

5.3. Comparative Detection Time

Tables 4 and 5 present an overview of the outcomes of tests conducted with different machine learning methods. Testing Time is the length of time needed to use 5-fold cross-validation to classify the testing dataset; Model Time is the processing time required to generate the machine learning models; and Accuracy is the classification accuracy reached by the various methodologies. The classification accuracy is illustrated in Figures 3 and 4.

The testing time in Tables 4 and 5 represents the execution time needed by each model for classifying incoming SQL queries during the deployment. Measuring testing time is important for real-time applications, where detection must happen within the normal query processing time frame. The evaluated models' results achieve low execution times, revealing the ability to perform effectively in real-time environments. Also, the presented execution times are considered sufficiently small compared to typical query timing in web applications, while maintaining their practical applicability in real-time environments.

Table 4 offers a comparison of detection times across various classifiers using a UCI dataset. Classifiers are algorithms utilized in machine learning to predict categorical labels for given inputs, with the UCI dataset comprising a repository of databases commonly employed in machine learning and data mining research.

Observing Table 4 and Figure 3, SVM exhibits the highest mean accuracy of 0.9808, closely trailed by XGBoost with a mean accuracy of 0.9705. While Light Gradient Boost yields a mean accuracy of 0.9649, falling short of SVM and XGBoost, it demonstrates shorter durations for both training and prediction in comparison to the classifiers. In contrast, AdaBoost records the lowest mean accuracy at 0.6661 and demands the longest durations for both training and prediction among all classifiers listed in the table.

Table 4. Detection time comparison of machine learning classifiers (SVM, XGBoost, AdaBoost, and LightGBM) on the SQL injection dataset (UCI repository), showing the time required for model evaluation.

Classifiers	Mean Accuracy	Model Time	Testing Time
SVM	0.9808	2 min 35.80 s	30.60 s
XGBoost	0.9705	3 min 30.85 s	33.50 s
Light Gradient Boost	0.9649	1 min 42.65 s	35.45 s
AdaBoost	0.6661	5 min 30.20 s	1 min 10.45 s

Table 5. Detection time comparison of machine learning classifiers (SVM, XGBoost, AdaBoost, and LightGBM) on the Kaggle SQL injection dataset, representing the testing time required for model evaluation.

Classifiers	Mean Accuracy	Model Time	Testing Time
SVM	0.9847	2 min 18.70 s	29.10 s
XGBoost	0.9749	3 min 10.40 s	31.20 s
Light Gradient Boost	0.9654	1 min 17 s	35 s
AdaBoost	0.84725	4 min 50.90 s	59.80 s

On the other hand, Table 5 and Figure 4 compare the detection time of four machine learning classifiers (SVM, XGBoost, Light Gradient Boost, and AdaBoost) on a dataset from Kaggle in 2020. The comparison is based on two factors: model time and testing time. SVM has the fastest testing time (29.10 s). Based on this table, SVM appears to be the fastest

classifier for this dataset in terms of model time and testing time. However, based on the particulars of the problem being solved, a classifier will be selected.

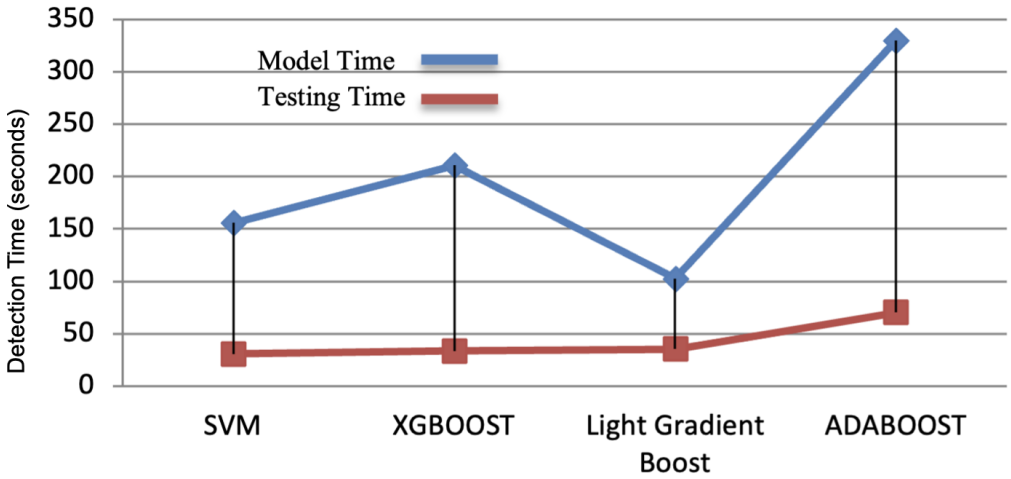

Figure 3. Comparison of detection time for different classifiers on the UCI dataset. The X-axis represents the evaluated classifiers (SVM, XGBoost, AdaBoost, and LightGBM), while the Y-axis represents detection time measured in seconds.

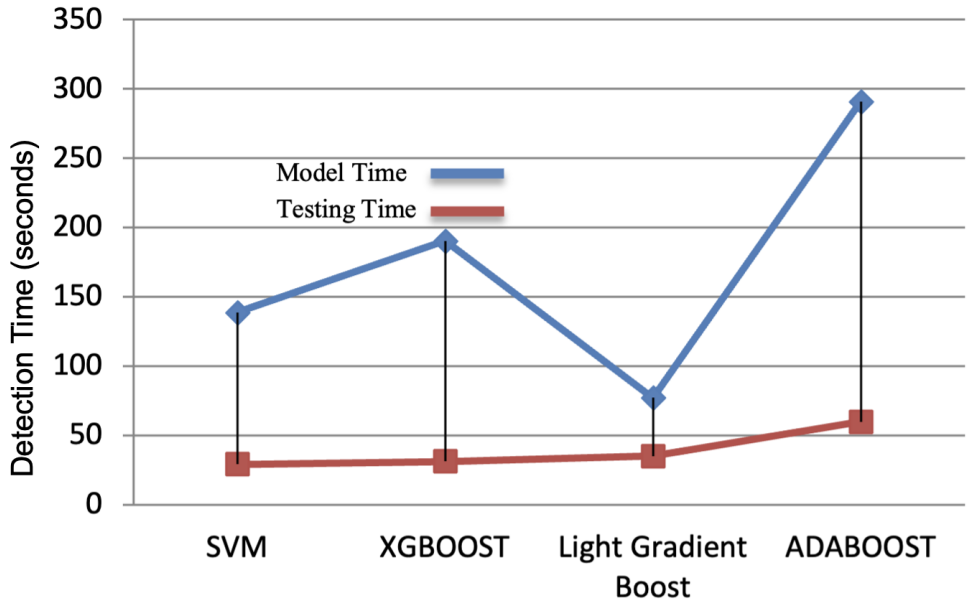

Figure 4. Comparison of detection time for different classifiers on the Kaggle SQL injection dataset. The X-axis represents the evaluated classifiers (SVM, XGBoost, AdaBoost, and LightGBM), while the Y-axis represents detection time measured in seconds.

The evaluated classifiers encompass Support Vector Machine (SVM), XGBoost, Light Gradient Boost, and AdaBoost. The table furnishes insights into their accuracy, model time, and testing time. Reflecting the classifier's capability to accurately categorize instances within the dataset, SVM attained the highest accuracy score of 0.9847, denoting a correct classification rate of 98.47%. This denotes the duration required to train the classifier on the dataset. AdaBoost recorded the longest model time of 4 min 50.90 s, indicating a prolonged training duration. The high accuracy score of SVM positions it as a favorable choice for this dataset, further accentuated by its relatively short model and testing times. XGBoost, with accuracy close to SVMs, serves as a viable alternative. Nonetheless, its longer model time may pose a limitation in certain applications. Despite Light Gradient Boost's lower

accuracy score compared to the top classifiers, its short model time renders it an appealing option, particularly where speed is paramount. AdaBoost's notably low accuracy score and prolonged model and testing times render it the least preferable choice for this dataset.

The training time is not a crucial constraint for real-time deployment because the model training is typically done offline and does not need regular updates, as models can be periodically retrained or when new attack patterns appear. On the other hand, detection time is more related to real-world applications, which directly affects the application's ability to detect SQLI attacks in real time. The results reveal that all evaluated models are suitable for deployment in web-based security techniques because they have achieved low detection times.

In summary, the selection of the classifier hinges on project-specific requirements. If accuracy takes precedence, SVM emerges as the optimal choice, whereas Light Gradient Boost may be preferable for applications prioritizing speed. However, it is imperative to conduct a comprehensive analysis of the dataset and project prerequisites to make an informed decision regarding the choice of classifier. Although the SVM model achieved the highest accuracy, compared to other models, the differences are somewhat small; therefore, the outcomes should be viewed as indicative rather than definitive. The high performance of the SVM model can be attributed to a number of factors related to the data characteristics and the feature representation. First, using the TF-IDF results in high-dimensional and sparse feature vectors, which are considered well-suited for an SVM model, as it works well with such data representations and recognizes the best decision boundaries. The second factor is that the SQL queries show structured patterns and keyword distributions between benign and malicious queries. SVM is particularly able to detect these patterns by maximizing the margin between classes, allowing generalization. Additionally, SVM is less sensitive to overfitting and noise, which may help improve the model's performance [55].

5.4. Comparative Result with Previous Works

In the Section 5, the author delves into the application of machine learning algorithms for detecting SQL injection attacks, specifically SVM, XGBoost, Light Gradient Boost, and AdaBoost. The study outlines the rationale behind selecting these algorithms, highlighting their respective strengths and weaknesses in the context of SQL injection attack detection.

Table 6 summarizes previous ML approaches for SQL injection detection. Even though previous studies used deep learning or ensemble models, the need for systematic comparisons of traditional machine learning algorithms using consistent preprocessing techniques and datasets remains.

Comparing the results of this study with the performance of ML models in previous studies may differ because of the differences in the dataset (size, balance, and complexity), feature extraction approaches, preprocessing steps, and evaluation metrics. As presented in Table 6, some of the studies report lower performance for SVM, while our study reports high performance for SVM under specific conditions, specifically with the use of TF-IDF feature representation and a unified preprocessing pipeline.

Accordingly, the reported performance values are not strictly comparable, and differences in evaluation metrics (accuracy) do not necessarily imply that any single model consistently outperforms others across all scenarios. Instead, the table delivers a general overview of trends in previous studies, while this study's outcomes present a controlled and standardized evaluation of models under the same experimental framework.

Table 6. Comparative summary of representative studies on SQL injection detection, including classifiers, feature extraction techniques, and reported performance metrics.

Reference	Classifier(s)	Feature Extraction	Metric	Value
[56]	JRip, J48, RF, SVM, ANN	Features extracted from SQL queries and web traffic; attributes selected from network and database logs	Accuracy	SVM: 0.9403 ANN: 0.9672 J48: 0.9563 RF: 0.9653 JRip: 0.9474
[57]	Regex, Gradient Boosting, Naïve Bayes, SVM	Pattern-based SQL injection detection using regular expressions; performance improvement via TPR and TNR optimization	Accuracy /Precision	Accuracy: 0.9700 Precision: 1.0000
[51]	GBM, AdaBoost, XGBoost, LightGBM	Tokenization and feature extraction from SQL query tokens	Accuracy	LightGBM: 0.9934 AdaBoost: 0.9911
[58]	KNN, Gradient Boosting, Logistic Regression, Naïve Bayes, SVM	NLP-based feature extraction from SQL queries	Accuracy	SVM: 0.9937 KNN: 0.9970 Logistic Regression: 0.9960 Gradient Boosting: 0.9948 Naïve Bayes: 0.9754
[59]	Logistic Regression, Random Forest, SVM, ANN, Decision Tree, CNN, Naïve Bayes	Feature extraction using RNN autoencoder; automatic feature learning	Accuracy/F1-score	Accuracy: 0.9400 F1-score: 0.9200

6. Conclusions

Detecting SQL injection attacks is crucial for preventing unauthorized access to sensitive data and ensuring the security of web applications. ML algorithms have shown great promise in identifying these attacks. By utilizing these algorithms, the accuracy of detecting SQLI attacks can be significantly increased. These models can learn patterns from data and detect anomalies that may indicate an attack. Effective ML algorithms, such as XGBoost, AdaBoost, SVM, and LightGBM, have been shown to accurately identify SQLI attacks by learning complex patterns in the data. Feature engineering is a critical step in this process, involving extracting relevant features, such as query syntax and semantics, to train the models. ML algorithms can also detect SQL injection attacks in real time, enabling rapid response to potential threats and minimizing damage. Combining these algorithms with traditional methods, such as rule-based systems, can further enhance detection accuracy. This study offered an ML-based approach to detect SQLI attacks utilizing a unified experimental framework in web applications. This study implemented and tested the following algorithms: XGBoost, AdaBoost, SVM, and Light Gradient Boost. Additionally, it utilized NLP to improve text processing detection accuracy; the primary dataset was transformed into a corpus to make feature engineering analysis easier. Two SQL injection datasets were used: the first for training and testing the classifier, and the second for evaluating its performance. Results indicated that the SVM algorithm achieved the highest detection accuracy, with a rate of 0.9808. In performance testing, the SVM classifier successfully detected 98.5% of the instances in the second dataset. The paper also reported the detection accuracy of the

other tested algorithms: XGBoost (0.9705), LightGBM (0.9649), and AdaBoost (0.8473). The results demonstrated that SVM achieved the best performance. Overall, ML algorithms have the potential to significantly improve the detection of SQL injection attacks and bolster the security of web applications. The main strength of this study is to enable a fair comparison and provide clearer insights into the effectiveness of the SQLI detection model. The added value of this study is its contribution to bridging the gap between traditional rule-based detection approaches and data-driven approaches by showing the effectiveness of ML techniques in identifying both known and potentially obfuscated attacks. The results of this study can help future work in enhancing web application security.

One limitation of this research is the dependency on a single dataset for training and evaluation. Future research may extend the framework by incorporating multiple datasets to improve the robustness of the proposed method. Another limitation is the use of a fixed train–test split without reporting statistical variability measures such as standard deviation across cross-validation folds. Future work will include statistical significance analysis to compare model performance more robustly.

7. Future Works

This study has shown how machine learning methods may be used to detect SQL injection attacks. However, there are still several areas that could benefit from further improvement and research. Potential future work includes:

Incorporating more data sources: Several techniques can be employed to enhance the detection of SQL injection attacks utilizing machine learning algorithms such as XGBoost, AdaBoost, SVM, and LightGBM. To increase model accuracy, feature engineering selects and modifies pertinent elements from input data, such as query duration, number of special characters, and certain keywords. By altering the available data—for example, by introducing noise to input searches or crafting new queries with related patterns—data augmentation can produce more training data. Model ensembling improves accuracy by combining predictions from multiple models using techniques like voting or stacking. Hyperparameter tuning, using methods like grid search or random search, optimizes model performance. Transfer learning leverages pre-trained models trained on similar datasets to enhance performance. Anomaly detection identifies unusual patterns indicating potential SQL injection attacks using techniques like one-class SVM or autoencoders. Lastly, adversarial training involves adding adversarial cases to the training set and strengthening models against adversarial attacks.

Assessing the model using a larger and more diverse dataset: The dataset should accurately reflect real-world scenarios, including a wide variety of SQL injection attacks and regular traffic with different degrees of complexity, to assess a model for detecting SQL injection attacks. To ensure that the model is evaluated on unseen data, the dataset should be divided into training, validation, and testing sets. Typically, this split is 70% training, 15% validation, and 15% testing.

Variability and Robustness Analysis: Cross-validation, such as k-fold, was employed in this study, in which the dataset has been divided into k subgroups, and the model is trained and tested k times, each time using a different subset as the test set. In future work, it should incorporate variability measures (e.g., standard deviation and confidence intervals) across folds to ensure robustness and prevent overfitting. Monitoring performance indicators like recall, accuracy, F1 score, and precision can provide a thorough understanding of the model's functioning. In real-world scenarios, class imbalance, where normal traffic instances outnumber SQL injection attacks, can bias the model toward predicting the majority class. This can be addressed using techniques like oversampling, undersampling, or generating synthetic samples.

Exploring other cybersecurity applications: DDoS attacks overwhelm websites or networks with traffic, making them inaccessible, while Advanced Persistent Threats (APTs) are multi-stage, sophisticated attacks designed to evade detection, and insider threats originate from within an organization. Machine learning algorithms can detect these threats by identifying anomalies in network traffic for DDoS attacks, unusual behavior for APTs, and potential insider threats. Training these algorithms requires collecting and labeling relevant data, such as network traffic or email data. Testing various algorithms, including neural networks, decision trees, and random forests, helps find the most effective approach for each attack type. Evaluating the model on a large, diverse dataset ensures it generalizes to new data, identifying weaknesses or biases. Real-world deployment demands interpretability and explainability, achievable through interpretable models like decision trees or techniques like LIME or SHAP. Collaborating with cybersecurity professionals and data scientists can deepen problem understanding and validate machine learning models, ensuring their relevance and effectiveness in detecting a wide range of cyber-attacks.

Hybrid and Ensemble-Based Detection Systems: Future work may consider exploring combining multiple ML models as hybrid approaches to capture complementary patterns in SQL queries, which could improve detection performance and robustness of the classifiers. Another future direction would be the analysis of the misclassified instances that may help find the common failure modes across different models. The results of the analysis will help to develop more resilient and efficient SQLI detection systems.

Author Contributions: Conceptualization, H.A. and N.A.; Methodology, H.A.; Software, H.A.; Validation, H.A.; Formal Analysis, H.A.; Investigation, H.A.; Resources, H.A.; Data Curation, H.A.; Writing—Original Draft Preparation, H.A.; Writing—Review and Editing, N.A.; Visualization, H.A.; Supervision, N.A.; Project Administration, N.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable, as the study does not involve human or animal subjects.

Informed Consent Statement: Not applicable.

Data Availability Statement: The dataset used in this study was compiled from publicly available sources and existing open-access datasets. In accordance with ethical guidelines and data-use policies, the dataset and trained models may be shared for research purposes upon request. To support reproducibility, all methodologies, preprocessing steps, and experimental settings are fully described in the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Pan, Y.; Sun, F.; Teng, Z.; White, J.; Schmidt, D.; Staples, J.; Krause, L. Detecting web attacks with end-to-end deep learning. *J. Internet Serv. Appl.* **2019**, *10*, 16. [CrossRef]
2. Pattewar, T.; Patil, H.; Patil, H.; Patil, N.; Taneja, M.; Wadile, T. Detection of SQL Injection Using Machine Learning: A Survey. *Int. J. Comput. Appl.* **2019**, *6*, 239–246.
3. Fang, Y.; Peng, J.; Liu, L.; Huang, C. WOVSQLI: Detection of SQL Injection Behaviors Using Word Vector and LSTM. In *Proceedings of the 2nd International Conference on Cryptography, Security and Privacy*; Association for Computing Machinery: New York, NY, USA, 2018. [CrossRef]
4. Chen, D.; Yan, Q.; Wu, C.; Zhao, J. SQL Injection Attack Detection and Prevention Techniques Using Deep Learning. *J. Phys. Conf. Ser.* **2021**, *1757*, 012055. [CrossRef]
5. Yan, R.; Xiao, X.; Hu, G.; Peng, S.; Jiang, Y. New Deep Learning Method to Detect Code Injection Attacks on Hybrid Applications. *J. Syst. Softw.* **2017**, *137*, 67–77. [CrossRef]

6. Ogini, P.; Taylor, E.; Nwiabu, N. A deep learning approach for the detection of structured query language injection vulnerability. *Int. J.* **2022**, *11*, 211–217.
7. Crespo-Martínez, I.S.; Campazas-Vega, A.; Guerrero-Higueras, Á.M.; Riego-DelCastillo, V.; Álvarez-Aparicio, C.; Fernández-Llamas, C. SQL injection attack detection in network flow data. *Comput. Secur.* **2023**, *127*, 103093. [CrossRef]
8. Olena, T.; Anastasiia, D.; Yuliia, L. Analysis of vulnerabilities and security problems of web applications. *Syst. Technol.* **2023**, *3*, 25–37. [CrossRef]
9. OWASP Foundation. OWASP Top 10: The Ten Most Critical Web Application Security Risks. 2021. Available online: https://owasp.org/www-project-top-ten/ (accessed on 10 January 2026).
10. MITRE Corporation. CWE-89: Improper Neutralization of Special Elements Used in an SQL Command ('SQL Injection'). Available online: https://cwe.mitre.org/data/definitions/89.html (accessed on 10 January 2026).
11. Pejo, B.; Kapui, N. SQLi Detection with ML: A data-source perspective. *arXiv* **2023**, arXiv:2304.12115. [CrossRef]
12. Abaimov, S.; Bianchi, G. A survey on the application of deep learning for code injection detection. *Array* **2021**, *11*, 100077. [CrossRef]
13. Lin, Y.; Liao, Y.; Zeng, W.; Wei, Y.; Chen, D.; Yuan, X.; Li, Y.; Erkan, U.; Toktas, A.; Zhang, C.; et al. 3D Non-degenerate Hyperchaos: Design, Analysis, and Application in Image Encryption. *IEEE Trans. Consum. Electron.* **2026**. [CrossRef]
14. Shar, L.K.; Tan, H.B.K. Predicting SQL injection and cross site scripting vulnerabilities through mining input sanitization patterns. *Inf. Softw. Technol.* **2013**, *55*, 1767–1780. [CrossRef]
15. Abdullayev, V.; Chauhan, A.S. SQL injection attack: Quick view. *Mesopotamian J. Cybersecur.* **2023**, *2023*, 30–34. [CrossRef]
16. Baklizi, M.; Atoum, I.; Hasan, M.A.S.; Abdullah, N.; Al-Wesabi, O.A.; Otoom, A.A. Prevention of Website SQL Injection Using a New Query Comparison and Encryption Algorithm. *Int. J. Intell. Syst. Appl. Eng.* **2023**, *11*, 228–238.
17. Augustine, N.; Sultan, A.B.M.; Osman, M.H.; Sharif, K. Application of Artificial Intelligence in Detecting SQL Injection Attacks. *JOIV Int. J. Inform. Vis.* **2024**, *8*, 2131. [CrossRef]
18. Faisal Fadlalla, F.; Elshoush, H.T. Input Validation Vulnerabilities in Web Applications: Systematic Review, Classification, and Analysis of the Current State-of-the-Art. *IEEE Access* **2023**, *11*, 40128–40161. [CrossRef]
19. Sermpinis, T. Web Application Hacking: Advanced SQL Injection and Data Store Attacks. *Hakin9 Magazine*, 19 April 2022. Available online: https://hakin9.org/download/web-application-hacking-advanced-sql-injection-data-store-attacks/ (accessed on 1 January 2026).
20. Kali Tutorials. Hacking Websites Using SQL Injection Manually. Available online: https://www.kalitutorials.net/2014/03/hacking-websites-using-sql-injection.html (accessed on 10 January 2026).
21. Liu, H.; Gegov, A.; Haig, E. *Rule Based Systems for Big Data: A Machine Learning Approach*; Springer: Berlin/Heidelberg, Germany, 2016. [CrossRef]
22. Tang, P.; Qiu, W.; Huang, Z.; Lian, H.; Liu, G. Detection of SQL injection based on artificial neural network. *Knowl.-Based Syst.* **2020**, *190*, 105528. [CrossRef]
23. Dasmohapatra, S.; Priyadarshini, S. A Comprehensive Study on SQL Injection Attacks, Their Mode, Detection and Prevention. In *Proceedings of Second Doctoral Symposium on Computational Intelligence: DoSCI 2021*; Springer: Singapore, 2022; pp. 617–632. [CrossRef]
24. Zhang, W.; Li, Y.; Li, X.; Shao, M.; Mi, Y.; Zhang, H.; Zhi, G. Deep Neural Network-Based SQL Injection Detection Method. *Secur. Commun. Netw.* **2022**, *2022*, 4836289. [CrossRef]
25. Abdulmalik, Y. An Improved SQL Injection Attack Detection Model Using Machine Learning Techniques. *Int. J. Innov. Comput.* **2021**, *11*, 53–57. [CrossRef]
26. Shwaish, A.K.; Hussain, M.A.; Al-Kashoash, H.A. Encoding Query-Based Lightweight Algorithm for Preventing SQL Injection Attack. *Comput. Commun.* **2020**, *46*, 1–11. Available online: https://faculty.uobasrah.edu.iq/uploads/publications/1631190617.pdf (accessed on 5 May 2026).
27. Hasan, M.; Balbahaith, Z.; Tarique, M. Detection of SQL Injection Attacks: A Machine Learning Approach. In *Proceedings of the 2019 International Conference on Electrical and Computing Technologies and Applications (ICECTA)*; IEEE: Piscataway, NJ, USA, 2019; pp. 1–6. [CrossRef]
28. Wang, Y.; Li, Z. SQL Injection Detection via Program Tracing and Machine Learning. In *Proceedings of the Internet and Distributed Computing Systems*; Springer: Berlin/Heidelberg, Germany, 2012; Volume 7646, pp. 550–559. [CrossRef]
29. Kar, D.; Sahoo, A.K.; Agarwal, K.; Panigrahi, S.; Das, M. Learning to Detect SQL Injection Attacks Using Node Centrality with Feature Selection. In *Proceedings of the 2016 International Conference on Computing, Analytics and Security Trends (CAST)*; IEEE: Piscataway, NJ, USA, 2016; pp. 327–332. [CrossRef]
30. Zhang, K. A Machine Learning Based Approach to Identify SQL Injection Vulnerabilities. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*; IEEE: Piscataway, NJ, USA, 2019; pp. 1286–1288. [CrossRef]
31. Li, Q.; Wang, F.; Wang, J.; Li, W. LSTM-Based SQL Injection Detection Method for Intelligent Transportation System. *IEEE Trans. Veh. Technol.* **2019**, *68*, 4182–4191. [CrossRef]

32. Pathak, R.; Mohit; Yadav, V. Handling SQL Injection Attack Using Progressive Neural Network. In *Proceedings of the International Conference on Information, Communication and Computing Technology*; Springer: Singapore, 2020; pp. 231–241. [CrossRef]
33. Ahmed, M.; Uddin, M. Cyber Attack Detection Method Based on NLP and Ensemble Learning Approach. In *Proceedings of the 2020 23rd International Conference on Computer and Information Technology (ICCIT)*; IEEE: Piscataway, NJ, USA, 2020; pp. 1–6. [CrossRef]
34. Tripathy, D.; Gohil, R.; Halabi, T. Detecting SQL Injection Attacks in Cloud SaaS using Machine Learning. In *Proceedings of the 2020 IEEE 6th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing, (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)*; IEEE: Piscataway, NJ, USA, 2020; pp. 145–150. [CrossRef]
35. Hu, J.; Zhao, W.; Cui, Y. A Survey on SQL Injection Attacks, Detection and Prevention. In *Proceedings of the 2020 12th International Conference on Machine Learning and Computing*; Association for Computing Machinery: New York, NY, USA, 2020; pp. 483–488. [CrossRef]
36. Mejia-Cabrera, H.I.; Paico-Chileno, D.; Valdera-Contreras, J.H.; Tuesta-Monteza, V.A.; Forero, M.G. Automatic Detection of Injection Attacks by Machine Learning in NoSQL Databases. In *Proceedings of the Mexican Conference on Pattern Recognition*; Springer: Cham, Switzerland, 2021. [CrossRef]
37. Alqhtani, M.; Alghazzawi, D.; Alarifi, S. Detection of SQL Injection Attack Using Machine Learning Techniques: A Systematic Literature Review. *J. Cybersecur. Priv.* **2022**, *2*, 764–777. [CrossRef]
38. Muhammad, T.; Ghafory, H. SQL Injection Attack Detection Using Machine Learning Algorithm. *Mesopotamian J. Cyber Secur.* **2022**, *2022*, 5–17. [CrossRef]
39. Alarfaj, F.; Khan, N. Enhancing the Performance of SQL Injection Attack Detection through Probabilistic Neural Networks. *Appl. Sci.* **2023**, *13*, 4365. [CrossRef]
40. Lakshmi, D.B.S.; Kovvuri, D.; Bolisetti, V.; Chikkala, D.S.; Karri, S.; Yadlapalli, G. A Proactive Approach for Detecting SQL and XSS Injection Attacks. In *2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*; IEEE: Piscataway, NJ, USA, 2024; pp. 1415–1420. [CrossRef]
41. Thilakraj, M.; Anupriya, S.; Cibi, M.; Divya, A. Detection of SQL injection attacks. In *Proceedings of the 2024 International Conference on Inventive Computation Technologies (ICICT)*; IEEE: Piscataway, NJ, USA, 2024; pp. 1515–1520. [CrossRef]
42. Al Mallah, R.; Quintero, A. Adversarial Threats and Defense Mechanisms in Machine Learning-Based SQL Injection Detection: A Security Analysis. In *2025 International Conference on Computing, Networking and Communications (ICNC)*; IEEE: Piscataway, NJ, USA, 2025; pp. 180–184. [CrossRef]
43. Hussain, S.S. SQL Injection Dataset. Kaggle Dataset. 2022. Available online: https://www.kaggle.com/syedsaqlainhussain/sql-injection-dataset (accessed on 3 September 2024).
44. Graff, C.; Graff, D. UCI Machine Learning Repository. 2019. Available online: http://archive.ics.uci.edu/ml (accessed on 3 September 2024).
45. Dini, P.; Elhanashi, A.; Begni, A.; Saponara, S.; Zheng, Q.; Gasmi, K. Overview on intrusion detection systems design exploiting machine learning for networking cybersecurity. *Appl. Sci.* **2023**, *13*, 7507. [CrossRef]
46. Wagner, C.; François, J.; State, R.; Engel, T. Machine Learning Approach for IP-Flow Record Anomaly Detection. In *Proceedings of the IFIP Networking Conference*; Springer: Berlin/Heidelberg, Germany, 2011; pp. 28–39. [CrossRef]
47. Yuan, R.; Li, Z.; Guan, X.; Xu, L. An SVM-based machine learning method for accurate Internet traffic classification. *Inf. Syst. Front.* **2010**, *12*, 149–156. [CrossRef]
48. Ahmed, S.; dabag, M. Machine Learning and Deep Learning Approaches for SQL Injection Detection: A Review. *NTU J. Eng. Technol.* **2025**, *4*, 1–17. [CrossRef]
49. AL-Maliki, M.H.A.; Jasim, M.N. Review of SQL injection attacks: Detection, to enhance the security of the website from client-side attacks. *Int. J. Nonlinear Anal. Appl.* **2022**, *13*, 3773–3782.
50. M R, P.; Giri, J.; Chadge, R.; Sunheriya, N.; Mahatme, C.; mallik, S.; Alsubaie, N.; Alqahtani, M.; Abbas, M.; Othman, S. An Optimized Machine Learning Model for the Early Prognosis of Chronic Respiratory Diseases with specific focus On Asthma. *Res. Sq.* **2023**. [CrossRef]
51. Farooq, U. Ensemble machine learning approaches for detection of SQL injection attack. *Teh. Glas.* **2021**, *15*, 112–120. [CrossRef]
52. Mahmood, S. SQL Injection Detection Using Machine Learning and Explainability. *J. Internet Serv. Inf. Secur.* **2025**, *15*, 309–324. [CrossRef]
53. Scikit-Learn. Feature Selection. 2022. Available online: https://scikit-learn.org/stable/modules/feature_selection.html (accessed on 3 September 2024).
54. Pfob, A.; Lu, S.C.; Sidey-Gibbons, C. Machine learning in medicine: A practical introduction to techniques for data pre-processing, hyperparameter tuning, and model comparison. *BMC Med. Res. Methodol.* **2022**, *22*, 282. [CrossRef]
55. Hariyani, A.; Dolia, P. CryptoSQLShield: A Comprehensive Study on Cryptography-Assisted Methods for SQL Injection Defense. *Int. J. Eng. Res. Technol.* **2026**, *15*, 1–13.

56. Ross, K.; Moh, M.; Moh, T.S.; Yao, J. Multi-source data analysis and evaluation of machine learning techniques for SQL injection detection. In *Proceedings of the of the ACMSE 2018 Conference*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 1–8. [CrossRef]
57. Kranthikumar, B.; Velusamy, R.L. SQL Injection Detection Using REGEX Classifier. *Int. J. Adv. Comput. Sci. Appl.* **2020**, *12*, 800–809.
58. Triloka, J.; Hartono, H.; Sutedi, S. Detection of SQL Injection Attack Using Machine Learning Based on Natural Language Processing. *Int. J. Artif. Intell. Res.* **2022**, *6*, 355. [CrossRef]
59. Alqhtani, M.; Alghazzawi, D.; Alarifi, S. Deep Learning Architecture for Detecting SQL Injection Attacks Based on RNN Autoencoder Model. *Mathematics* **2023**, *11*, 3286. [CrossRef]